

SISTEMAS OPERATIVOS

Conferencia 3 · Tema II: Sincronización y Exclusión Mutua

Carrera: Ingeniería Informática	Profesora: Ing. Claudia Vecino Fernández
Año: 3ro Período: 2do Tipo de curso: CRD	Correo: claudiavf@uho.edu.cu
Disciplina: Infraestructura de Sistemas Informáticos	Teléfono: 54196249

Objetivo: Comprender el problema de la sección crítica, los mecanismos de exclusión mutua con espera ocupada y con bloqueo, el uso de semáforos como primitiva de sincronización, y aplicar estos conceptos a los problemas clásicos de sincronización.

Sumario

- Sincronización entre procesos. Condiciones de competencia.
- Sección crítica y exclusión mutua.
- Algoritmos con espera ocupada: alternancia, banderas, Peterson.
- Instrucciones hardware: test-and-set, swap.
- Exclusión mutua con bloqueo: semáforos y primitivas P / V (Down / Up).
- Semáforos con estructura de bloqueo. Implementación en C.
- Problemas clásicos de sincronización: Productor/Consumidor, Lectores/Escritores.

Bibliografía

- Tanenbaum, A. S. Sistemas Operativos Modernos. Págs. 38–70.
- Tanenbaum, A. S. Operating System: Design and Implementation. Págs. 51–67.
- Peterson, J. y Silberschatz, A. Operating System Concepts. Págs. 326–347.
- Silberschatz, Galvin, Gagne. Operating System Concepts, 10ª ed. Caps. 6–7.

1. Sincronización entre Procesos. Condiciones de Competencia

En el ejemplo del productor/consumidor ambos procesos necesitan esperar cuando ocurren condiciones extremas (buffers llenos y vacíos, respectivamente). La forma de esperar mediante un lazo donde no se hace nada se llama espera ocupada (busy waiting).

La espera ocupada puede no ser un mecanismo adecuado. En un sistema con un solo procesador, si el CPU está siendo utilizado por el proceso que está en espera, el otro proceso no podrá tomarlo y no se saldrá de esa situación. A esta situación se le conoce como Abrazo fatal (deadlock).

Definición: Las condiciones de competencia (race conditions) son situaciones en las cuales dos o más procesos acceden en forma simultánea a datos compartidos, y el resultado del procesamiento depende del orden en que las instrucciones se ejecuten. No

es posible hablar de procesos adecuadamente sincronizados si no se ha evitado la ocurrencia de estas condiciones.

Tomemos el ejemplo del productor/consumidor. En el productor aparece la instrucción `counter++` y en el consumidor `counter--`. Aplicando las condiciones de Bernstein:

Variable	R(producer) / W(producer)
counter++	R: {counter} W: {counter}
counter--	R: {counter} W: {counter}

Como $R(S1) \cap W(S2) = \{\text{counter}\} \neq \emptyset$, las instrucciones no pueden ejecutarse concurrentemente. Si ocurriera, `counter` podría tomar el valor incorrecto de $m+1$ o $m-1$ en lugar de m .

1.1 De la Espera Ocupada al Bloqueo

La solución real consiste en que cuando un proceso necesita esperar se bloquee: se le retira el CPU y se le pone en una cola de espera hasta que ocurra el evento deseado. Para ello se usan dos primitivas básicas:

Primitiva	Descripción
<code>sleep() / block()</code>	El proceso que la ejecuta pasa al estado Bloqueado.
<code>wake_up(proceso)</code>	Pasa el proceso indicado del estado Bloqueado al estado Listo.

Con estas primitivas la solución del productor/consumidor garantiza que un proceso despierte al otro cuando haya producido o consumido un buffer:

```
/* Productor con bloqueo */
void productor() {
    while(1) {
        producir_elemento(&nextp);
        if (counter == N) sleep(); // buffer lleno → bloquearse
        buffer[in] = nextp;
        in = (in + 1) % N;
        counter++;
        if (counter == 1) wake_up(consumidor); // avisar al consumidor
    }
}

/* Consumidor con bloqueo */
void consumidor() {
    while(1) {
        if (counter == 0) sleep(); // buffer vacío → bloquearse
        nextc = buffer[out];
        out = (out + 1) % N;
        counter--;
        if (counter == N-1) wake_up(productor); // avisar al productor
        consumir_elemento(nextc);
    }
}
```

⚠ Importante: Esta solución tiene una condición de carrera sutil: si el consumidor comprueba `counter==0` y antes de ejecutar `sleep()` pierde el CPU, el productor ejecuta `wake_up()` sobre un proceso que aún no está dormido, perdiéndose la señal. Al retornar, el consumidor duerme indefinidamente → deadlock. Esta es la motivación para los semáforos.

2. Sección Crítica y Exclusión Mutua

Definición: La sección crítica de un proceso es el segmento de código que accede a variables, tablas, ficheros y otros elementos compartidos con otros procesos. La ejecución de las secciones críticas de los distintos procesos debe realizarse de forma mutuamente exclusiva.

Un proceso se estructura en cuatro segmentos respecto a su sección crítica:

Segmento	Descripción
Sección de entrada	Solicita permiso para entrar en la sección crítica.
Sección crítica	Código que accede a recursos compartidos.
Sección de salida	Habilita que otro proceso pueda entrar en su sección crítica.
Sección restante	El resto del código del proceso, sin acceso a recursos compartidos.

2.1 Requisitos de una Solución Correcta

Para que un método garantice la exclusión mutua y logre un procesamiento paralelo correcto deben cumplirse cuatro condiciones fundamentales:

Requisito	Descripción
Exclusión mutua	No pueden estar simultáneamente dos procesos en sus respectivas secciones críticas.
Progreso	La solución no puede basarse en suposiciones sobre velocidades relativas de los procesos. Ningún proceso fuera de su sección crítica puede impedir que otro entre en la suya.
Espera acotada (Bounded waiting)	Ningún proceso deberá esperar un tiempo arbitrariamente largo para entrar en su sección crítica (evitar inanición, starvation).
Independencia del hardware	La solución no puede depender del número de procesadores ni de sus velocidades relativas.

🔴 Nota actualizada: Estos cuatro requisitos fueron formalizados por Dijkstra en 1965 y siguen siendo la base del diseño de mecanismos de sincronización en sistemas operativos modernos como Linux, Windows y macOS.

3. Exclusión Mutua con Espera Ocupada

Se presentan a continuación los principales algoritmos de exclusión mutua con espera ocupada (busy waiting), considerando dos procesos P_i y P_j (por ejemplo P_0 y P_1).

3.1 Alternancia Estricta

Se basa en una variable entera común (turn) que indica cuál proceso tiene derecho a ejecutar su sección crítica. Si $\text{turn} = i$, el proceso P_i puede entrar.

```
short int turn;    // variable compartida, inicialmente: turn = i

/* Proceso  $P_i$  */
while (turn != i) ;    // sección de entrada: espera ocupada
seccion_critica();
turn = j;              // sección de salida: cede el turno
seccion_restante();
```

Propiedad	¿Se cumple?
Exclusión mutua	✓ Sí — solo un proceso entra cuando $\text{turn} = i$.
Progreso	✗ No — exige alternancia estricta. Si P_i no quiere entrar, P_j tampoco puede.
Espera acotada	✓ Sí — cada proceso espera como máximo un turno del otro.

3.2 Algoritmo con Banderas (Flags)

Cada proceso mantiene una bandera que indica si está interesado en entrar. El proceso P_i espera mientras el otro tenga su bandera activada.

```
short int flag[2] = {0, 0};    // banderas, inicialmente en 0

/* Proceso  $P_i$  */
while (flag[j]) ;              // sección de entrada: espera
flag[i] = TRUE;                // indico intención de entrar
seccion_critica();
flag[i] = FALSE;               // sección de salida
seccion_restante();
```

Propiedad	¿Se cumple?
Exclusión mutua	✗ No — si ambos leen $\text{flag}[j]=0$ al mismo tiempo, los dos entran.
Progreso	✗ No — variante con banderas al inicio provoca deadlock.
Espera acotada	Variable — depende de la variante.

3.3 Algoritmo de Peterson ★

Combina la variable `turn` con las banderas. Es el primer algoritmo que satisface completamente los cuatro requisitos para dos procesos.

```
short int turn;
short int flag[2] = {0, 0};

/* Proceso Pi - sección de entrada */
flag[i] = TRUE;           // manifiesto intención
turn = j;                 // cedo el turno al otro
while (flag[j] && turn == j) ; // espero si el otro quiere Y es su turno

seccion_critica();

/* Sección de salida */
flag[i] = FALSE;
seccion_restante();
```

Propiedad	¿Se cumple?
Exclusión mutua	✓ Sí — solo uno pasa el <code>while</code> .
Progreso	✓ Sí — si P_i no quiere entrar ($flag[i]=FALSE$), P_j entra libremente.
Espera acotada	✓ Sí — P_j espera como máximo un turno de P_i .
Independencia del hardware	✓ Sí — funciona con un solo procesador.

Un proceso entra en su sección crítica cuando se cumple alguna de estas condiciones: el turno le corresponde, o el turno es del otro proceso pero éste no está interesado ($flag[j]=FALSE$). Si ambos intentan entrar simultáneamente, solo el que no asignó $turn = j$ será el último en actualizar `turn` y podrá pasar.

✦ **Nota actualizada:** El algoritmo de Peterson es correcto para 2 procesos, pero escalar a n procesos requiere versiones más complejas (algoritmo de la panadería de Lamport). En arquitecturas modernas con reordenamiento de instrucciones (CPU out-of-order), se necesitan además barreras de memoria (memory barriers / fences) para garantizar la visibilidad de las escrituras.

3.4 Instrucciones Hardware para Exclusión Mutua

Muchas arquitecturas ofrecen instrucciones atómicas que simplifican la exclusión mutua, ejecutando lectura y escritura como una sola operación no interrumpible.

3.4.1 Instrucción Test-and-Set

Lee el valor de una variable, la pone a `TRUE` y devuelve el valor original, todo en un ciclo de memoria indivisible.

```
// Semántica de test_and_set (pseudocódigo)
bool test_and_set(bool *var) {
    bool temp = *var;
    *var = TRUE;
    return temp;
}
```

```

}

// Uso con variable de cierre (lock = FALSE inicialmente)
while (test_and_set(&lock)) ; // sección de entrada
seccion_critica();
lock = FALSE;                // sección de salida
seccion_restante();

```

3.4.2 Instrucción Swap

Intercambia atómicamente los contenidos de dos palabras de memoria. Cada proceso usa una variable local `key = TRUE`.

```

// Semántica de swap (pseudocódigo)
void swap(bool *a, bool *b) {
    bool temp = *a; *a = *b; *b = temp; // atómico
}

// Uso: lock = FALSE inicialmente, key = TRUE (local)
do { swap(&lock, &key); } while (key == TRUE);
seccion_critica();
lock = FALSE;
seccion_restante();

```

Instrucción	Exclusión mutua	Espera acotada
test_and_set simple	✓ Sí	X No (posible starvation)
swap simple	✓ Sí	X No (posible starvation)
Variantes con espera acotada	✓ Sí	✓ Sí (requiere arreglo adicional)

🔴 **Nota actualizada:** En arquitecturas modernas (x86-64, ARM), las instrucciones equivalentes son LOCK XCHG, CMPXCHG (Compare-and-Swap, CAS) y LDREX/STREX. Son la base de las operaciones atómicas en lenguajes de alto nivel: `std::atomic<>` en C++11, `AtomicInteger` en Java, y `sync/atomic` en Go.

4. Exclusión Mutua con Bloqueo: Semáforos

Los algoritmos con espera ocupada malgastan tiempo de CPU cuando un proceso no puede entrar en su sección crítica. La alternativa es bloquear el proceso hasta que pueda operar, usando operaciones atómicas denominadas primitivas. Los semáforos, propuestos por Dijkstra (1965), son el mecanismo clásico para esto.

Definición: Un semáforo S es una variable entera que, aparte de su inicialización, solo puede ser accedida a través de las operaciones atómicas P (Down / Wait) y V (Up / Signal). $P(S)$: decrementa S ; si $S < 0$, bloquea el proceso. $V(S)$: incrementa S ; si hay procesos bloqueados, despierta uno.

4.1 Definición Clásica vs. Implementación con Bloqueo

Aspecto	Definición clásica (espera ocupada)
P(S) — Down	while (S <= 0) ; S--; (no bloquea, espera activa)
V(S) — Up	S++; (simplemente incrementa)
Valor mínimo	$S \geq 0$ siempre
Uso de CPU	Alto cuando hay contención

Aspecto	Implementación con bloqueo (práctica)
P(S) — Down	S.value--; si S.value < 0: añadir proceso a la lista y bloquearlo.
V(S) — Up	S.value++; si S.value ≤ 0: retirar un proceso de la lista y despertarlo.
Valor mínimo	S.value puede ser negativo; S.value = nº de procesos bloqueados.
Uso de CPU	Bajo: proceso bloqueado no consume CPU.

4.2 Estructura e Implementación en C

Para implementar semáforos con bloqueo se requiere una estructura que combine un valor entero con una lista de procesos bloqueados en él:

```
#define NIL (PCB*) 0
#define N 30 /* Longitud de la tabla de procesos */

typedef struct {
    int value; /* Valor del semáforo */
    PCB *listhead; /* Cabeza de la lista de bloqueados */
    PCB *listtail; /* Cola de la lista de bloqueados */
} semaphore;

/* Operación P (Down / Wait) */
void P(semaphore *s) {
    lock(); /* Deshabilitar interrupciones */
    s->value--;
    if (s->value < 0) {
        añadir_proceso_a_lista(s, proc_actual);
        planificador(); /* Ceder el CPU */
    } else {
        unlock();
    }
}

/* Operación V (Up / Signal) */
void V(semaphore *s) {
    lock();
    s->value++;
    if (s->value <= 0) {
        PCB *p = retirar_proceso_de_lista(s);
        pasar_a_listo(p); /* Despertar proceso */
    }
    unlock();
}
```

4.3 Uso de Semáforos para Exclusión Mutua

Con semáforos, la sección crítica de n procesos se implementa de forma sencilla usando un semáforo binario mutex inicializado en 1:

```
semaphore mutex;
mutex.value = 1;    /* inicialización */

/* Proceso Pi (cualquiera de los n procesos) */
P(&mutex);           /* sección de entrada → si mutex=0: bloqueo */
seccion_critica();
V(&mutex);           /* sección de salida → despierta a uno si hay
bloqueados */
seccion_restante();
```

4.4 Tipos de Semáforos

Tipo	Descripción y uso típico
Semáforo binario (mutex)	Toma solo los valores 0 y 1. Garantiza exclusión mutua en una sección crítica.
Semáforo contador (counting)	Toma valores enteros ≥ 0 . Controla el acceso a un recurso con N instancias disponibles (e.g., N buffers).

Las primitivas P y V pueden usarse también para programar cualquier grafo de precedencia, en conjunción con las instrucciones `parbegin/parend`: inicializar en 0 los semáforos de sincronización y ejecutar P antes del nodo dependiente y V tras el nodo del que depende.

🔴 **Nota actualizada:** En POSIX (Linux/macOS), los semáforos se usan mediante `sem_init()`, `sem_wait()` [$\equiv P$] y `sem_post()` [$\equiv V$]. En el núcleo de Linux se usan internamente `mutex` (`struct mutex`), `spinlocks` y `rwlocks`. En Java, la clase `java.util.concurrent.Semaphore` ofrece `acquire()` y `release()`. En Python, `threading.Semaphore` provee `acquire()` y `release()`.

5. Problemas Clásicos de Sincronización

En la literatura se recogen numerosos problemas de acciones concurrentes que requieren sincronización. Se presentan a continuación los más relevantes, resueltos con semáforos.

5.1 Problema Productor/Consumidor con Buffer Acotado

Dos procesos comparten un buffer circular de tamaño N . El productor añade elementos; el consumidor los extrae. Se necesita: (a) Exclusión mutua en el acceso al buffer. (b) Bloquear al productor cuando el buffer está lleno. (c) Bloquear al consumidor cuando el buffer está vacío.

Semáforo	Valor inicial	Significado
mutex	1	Exclusión mutua en el acceso al buffer compartido.

Semáforo	Valor inicial	Significado
full	0	Cuenta los buffers llenos (elementos disponibles para el consumidor).
empty	N	Cuenta los buffers vacíos (espacios disponibles para el productor).

```

/* Productor/Consumidor con semáforos */
semaphore mutex = {1}, full = {0}, empty = {N};
item buffer[N];
int in = 0, out = 0;

void productor() {
    while(1) {
        item nextp = producir_elemento();
        P(&empty);          // ¿hay espacio? si no → bloqueo
        P(&mutex);          // exclusión mutua
        buffer[in] = nextp;
        in = (in + 1) % N;
        V(&mutex);
        V(&full);           // avisar: hay un elemento más
    }
}

void consumidor() {
    while(1) {
        P(&full);           // ¿hay elementos? si no → bloqueo
        P(&mutex);          // exclusión mutua
        item nextc = buffer[out];
        out = (out + 1) % N;
        V(&mutex);
        V(&empty);          // avisar: hay un espacio más
        consumir_elemento(nextc);
    }
}

```

⚠ Importante: El orden de P(&empty)/P(&full) antes de P(&mutex) es crítico. Si se invirtiera (P(&mutex) primero), podría producirse un deadlock: el productor retiene mutex esperando empty, mientras el consumidor necesita mutex para liberar empty.

5.2 Problema de los Lectores/Escritores

Un almacén de datos (base de datos) es compartido entre múltiples procesos. Algunos solo leen (lectores) y otros actualizan los datos (escritores). Las reglas son:

- Pueden estar múltiples lectores accediendo simultáneamente (no se viola Bernstein entre lectores).
- Si un escritor está accediendo, ningún otro proceso (ni lector ni escritor) puede hacerlo.

Variante	Prioridad
1er problema (clásico)	Lectores — ningún lector espera a menos que un escritor tenga el acceso.
2do problema	Escritores — un escritor listo debe escribir tan pronto como sea posible.
Solución justa (fair)	Ninguna — orden de llegada (FIFO).

5.2.1 Primer Problema (Prioridad a Lectores)

```

/* Primer problema de lectores/escritores (prioridad lectores) */
semaphore mutex = {1}; // protege readcount
semaphore wrt   = {1}; // exclusión mutua en escritura
int readcount = 0;     // número de lectores activos

void lector() {
    while(1) {
        P(&mutex);
        readcount++;
        if (readcount == 1) P(&wrt); // primer lector bloquea escritores
        V(&mutex);
        leer_datos();
        P(&mutex);
        readcount--;
        if (readcount == 0) V(&wrt); // último lector libera a escritores
        V(&mutex);
    }
}

void escritor() {
    while(1) {
        elaborar_datos();
        P(&wrt); // acceso exclusivo
        escribir_datos();
        V(&wrt);
    }
}

// Inicialización: mutex.value=1; wrt.value=1; readcount=0;

```

🔴 **Nota actualizada:** En Linux, el problema de lectores/escritores se resuelve con `rwlock_t` (read-write spinlock) o `struct rw_semaphore`, que proporcionan `down_read()/up_read()` y `down_write()/up_write()`. En Java, `ReentrantReadWriteLock` ofrece `readLock()` y `writeLock()`. Desde Linux 5.x se prefieren `rwsem` con optimizaciones de equidad para evitar la inanición de escritores.

5.3 Comparativa de Mecanismos de Sincronización

Mecanismo	Espera ocupada	Bloqueo	n procesos	Moderno
Alternancia estricta	✓	X	X (2)	X
Banderas	✓	X	X (2)	X
Peterson	✓	X	✓ (extensible)	Parcial

Sistemas Operativos Conferencia 3			Ingeniería Informática · 3er Año	
Mecanismo	Espera ocupada	Bloqueo	n procesos	Moderno
Test-and-Set / Swap	✓	X	✓	✓ (CAS)
Semáforos P/V	Clásica	✓ (con bloqueo)	✓	✓ (POSIX)
Mutex/Futex	No	✓	✓	✓ (Linux)

6. Estudio Independiente

1. ¿Qué es una condición de competencia (race condition)? Ilustre con el ejemplo de counter++ / counter-- ejecutados concurrentemente.
2. Enumere los cuatro requisitos que debe cumplir una solución correcta al problema de la sección crítica.
3. Explique por qué el algoritmo de alternancia estricta no satisface el requisito de progreso. ¿Cómo lo corrige el algoritmo de Peterson?
4. ¿Cuál es la diferencia entre un semáforo binario y un semáforo contador? Proporcione un ejemplo de uso para cada uno.
5. En la solución del productor/consumidor con semáforos, explique por qué el orden P(&empty) antes de P(&mutex) es obligatorio en el productor.
6. Implemente en pseudocódigo el segundo problema de lectores/escritores (prioridad a escritores) usando semáforos P/V.
7. Investigue qué es un futex (fast userspace mutex) en Linux y en qué se diferencia de un semáforo POSIX. ¿En qué casos se prefiere uno sobre el otro?

— Fin de la Conferencia 3 —